

ISE– ISAD1000/5004 2024 Semester 1
ISE Assignment Marking Rubric

(The rubric is subject to change)

	Total marks for the section	Maximum marks allocation for sub sections
Demonstration 20 marks (The zip file submitted to Blackboard is to be downloaded to VDI and executed in the presence of the marker)		
Version Control	5	
– Created a Git repository as instructed and used it properly (including log, only few commits)		0.5
– Created several meaningful code commits at suitable time frames (code) and explained them		2
– Created several meaningful .md file commits at suitable time frames (docs) and explained them		1
– Meaningful use of branches/ merges and explained them		1.5
Modularity (Implementing Production Code)	5	
– Implementing the scenarios and running the production code		1
– Considered about good modularity principles, explained them correctly		3.5
– Overall completeness		0.5
Test Implementation	10	
– Tested different forms of imports/exports, explained the code /answered questions		3
– Tested different forms of data types, explained the code/answered questions		3
– Tested paths, explained the code/answered questions		2
– Set-up/tearing down, completeness, well explained		2
CODE 20 marks (Zip File Submission)		
Modularity (Production Code)	5	
– Readability (basic coding concepts, variable names, comments, etc.)		2
– Suitable module use and refactoring, if any		2
– Usefulness/overall functionality		1
Test Implementation (Test Code)	7	
– Implemented ALL designed test cases, compatibility with the design		1
– Testing different data types/import and export forms		4
– Setup and teardown		2
Version Control	8	
– .git folder, and suitable folder structure (doc/code, etc.)		3
– Git log/evidence of reasonable amount of commits in a suitable time frame for the code commits with suitable messages.		2
– Git log/evidence of reasonable amount of commits in a suitable time frame for the markdown (.md) file commits with suitable messages		2
– Evidence of appropriate branch merging		1
REPORT 60 marks (.md file/PDF Submitted in the .zip File and the Turnitin Submission)		
Cover page/section with ID, name, etc.	1	
Introduction/overview of the work done	1	
Module Description (Defining Suitable Modules to Solve a Simple Problem Using Sound Modularity Principles)	7	
– Defined modules in reasonable manner, several meaningful modules covering the given scenario		3
– Writing format, meaningful name, clear detailed explanation of the intended task, imports/exports, how input/outputs are handled (like keyboard input, etc.)		2
– Evidence/indication of how good modularity principles learned in the unit are applied in modules, any design decisions/revision evidence if any		2
Modularity (Implementing the Production Code)	10	
– Instruction to run the production code (Linux Environment, command line)		1

	Total marks for the section	Maximum marks allocation for sub sections
– Explanation on how modularity principles are used in the code (may use code snippets, screen shots)		3
– Modularity review checklist – (four or more reasonable items, proper descriptors)		2
– Use of the review checklist – Evidence of a reasonable attempt to use the checklist to properly check the production code (different modules)		3
– Discuss refactoring/or if not needed, justification as to why (may use code snippets, screen shots)		1
Test Design (Black Box)	16	
– Completeness/reasonable attempt to cover a particular module's functionality using EP approach with logical thinking, proper categories, test data, expected results		4
– Completeness/reasonable attempt to cover at least one module's functionality using BVA approach with logical thinking, proper categories, test data, expected results		2
– Specific test data: Given sample values (Last four digits of your student ID, your last name as it appears on your ID)		1
– Testing all modules (use EP to test all modules)/reasonable justification as to why if it has not been done so and covered in BVA		2
– Testing different data types (all three types: integer, string, Boolean)/import and export methods (all three forms: parameter passing and return values, keyboard input and screen output, file read/write)		6
– Explanation: brief explanation on why the tests were designed in the selected way /assumptions if any		1
Test Design (White Box)	9	
– Test design for paths of a suitable module: Reasonable idea of selecting test cases, covering all paths, test data, expected results		4
– Test design for paths of second module, second type of construct for paths (i.e. loop instead of if/else etc.): Reasonable idea of selecting test cases, covering all paths, test data, expected results		4
– Explanation: Brief explanation on why the tests were designed in the selected way		1
Test Implementation	3	
– Instructions on how to run the test code (Linux environment, command line)		1
– Results/output: Evidence of running tests, any assertion messages, etc. (with screenshots)		2
Workflow Table (Summary Table): Completed workflow table covering all modules, compatible with the work done under design, test implementation, and execution	5	
Version Control: Evidence of reasonable amount of commits for code, markdown (.md) file, use of branches based on the git log provided in the report, explanation on how the git section is handled	3	
Discussion: Reflection on own work (what you have achieved, challenges, limitations, ways to improve, etc.)	5	
TOTAL	100	
DEDUCTIONS		
<i>Not using Markdown (-30)</i>		
<i>Not converted to PDF (-5)</i>		
<i>Not done using Git (e.g. used GitHub or other tools) (-30)</i>		
<i>Not done in Linux/does not run in Linux command line (-50)</i>		
<i>Wrong zip file name (-5)</i>		
<i>Using programming languages other than Python/Java (coding section/s) will not be marked)</i>		
<i>Other issues will be handled case by case</i>		